1 Introdução

O computador pode ser pensado como uma calculadora super potente, capaz de realizar instruções quase que instantaneamente. Entretanto, para cada instrução executada, existe uma programação por trás. Por isso, saber uma linguagem de programação é essencial na área da computação. É através dela que conseguimos comunicar e determinar os comandos que o computador deve executar.

Dessa forma, podemos definir a linguagem de programação como um conjunto de instruções e regras que são usadas para criar programas. Programar pode ser comparado a escrever uma receita de bolo, onde devemos escrever um passo a passo bem claro e objetivo, para não haver erros. Um simples passo mal explicado é capaz de estragar a receita. Com as linguagens de programação, conseguimos desenvolver programas para diversos fins, que vão de jogos e aplicativos para celulares até a criação de sistemas autônomos na área da robótica.

Na aprendizagem de uma linguagem de programação, primeiro se entende os elementos da linguagem, depois se aprende códigos simples de apenas uma única linha, até chegar em um nível de programação onde é possível desenvolver um programa completo. Assim como é feito ao aprender um novo idioma. Quando começamos a aprender uma língua nova, primeiro aprendemos algumas palavras, depois frases pequenas e simples, até chegar em textos grandes e complexos. Por isso, antes de começar a estudar programação devemos estar cientes que aprender leva tempo e muita dedicação. Como tudo que é desconhecido, é muito comum ter dificuldade no início, mas com o passar do tempo, vai se tornando mais fácil.

1.1 Por que Python?

Python é uma linguagem de programação que foi criada em 1991 pelo holandês Guido Van Rossum, com o objetivo de ser uma linguagem simples e de fácil entendimento. Dessa forma, ela permite que o programador se concentre mais na resolução do problema em detrimento a gastar tempo com a programação em si. Além de ter uma sintaxe clara e concisa, Python é uma linguagem multiplataforma, funcionando muito bem em vários sistemas operacionais. Outra vantagem de se aprender esta linguagem e utilizá-la é que os recursos de extensibilidade dela permitem integrar componentes e invocar bibliotecas de outras linguagens.

Além das vantagens listadas acima, Python é uma linguagem que pode ser utilizada em diversas áreas, como na computação gráfica, para construir aplicativos de desktop e até jogos para dispositivos Android. Ela também é muito utilizada em *Big Data*, pois é possível processá-la facilmente, além de lidar com muitas dificuldades de dados e suportar computação paralela. Essa linguagem de programação também é bastante utilizada em aplicativos científicos, como o PyMOL, ferramenta para visualização e análise de estruturas moleculares tridimensionais, e o MPAlign editor, ferramenta para facilitar a análise e edição de seqüências moleculares (DNA/RNA/Proteína).

2 Instalação do ambiente Python

Antes de começar a programar, devemos instalar o interpretador da linguagem Python. O interpretador é o programa que traduz os algoritmos escritos para uma linguagem que o computador entenda e, dessa forma, ele executa as instruções, linha a linha. Se você utiliza o sistema operacional Linux ou Mac, o interpretador Python provavelmente já está instalado no seu computador. Dessa forma, você deverá verificar se está com a versão correta. Nós vamos utilizar a versão 3.6 ou 3.8. Já quem possui o Windows, deverá instalar fazendo o *download* da Internet.

2.1 Windows

Para instalar o interpretador é necessário acessar o *site* *www.python.org* e clicar nas seguintes abas: *Downloads* - Windows - Python 3.8.3 (ou a versão que estiver disponível), conforme mostra a Figura 1.

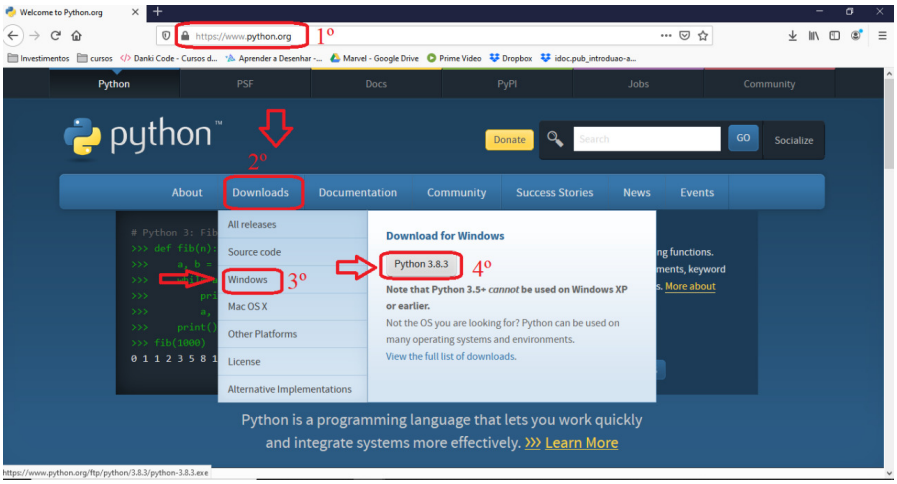

Figura 1: *Site* para *download* do interpretador Python no Windows.

Ao clicar em Python 3.8.3 abrirá uma janela para a confirmação de *download*, como mostra a figura abaixo.

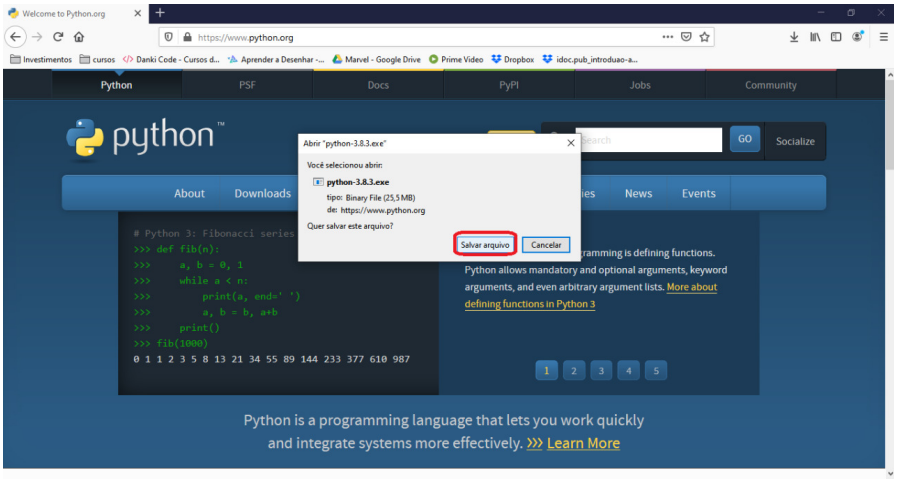

Figura 2: Janela para confirmação de *download*.

Após o *download* concluído é necessário abrir o arquivo pela aba de *downloads*, conforme a Figura 3.

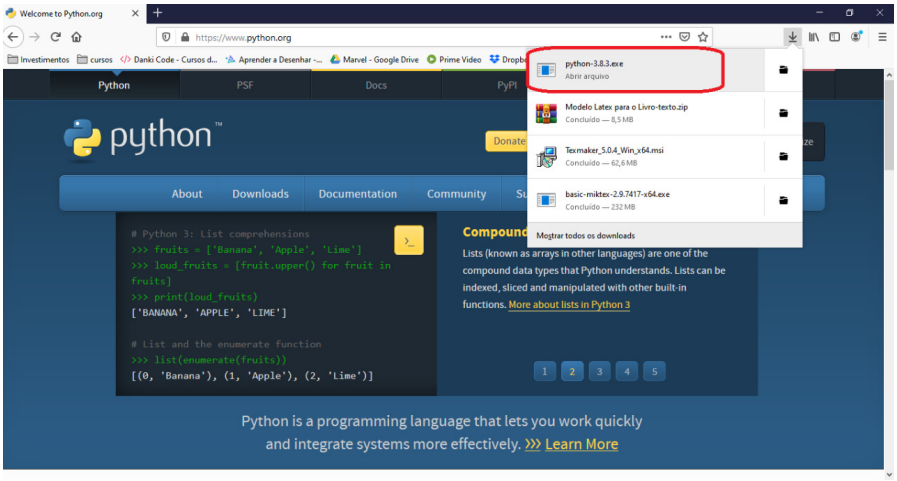

Figura 3: Aba de *downloads* concluídos no navegador Firefox.

Depois de aberto, no arquivo executável, clique em "*Install Now*", como na figura a seguir.

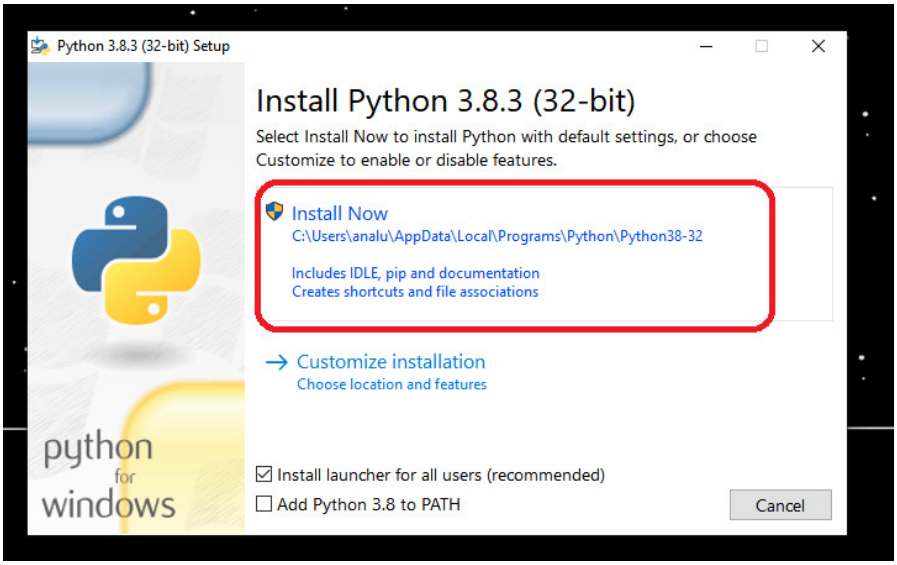

Figura 4: Janela de instalação Python.

Como é mostrado na Figura 5, quando a instalação for concluída, basta clicar em "*close*".

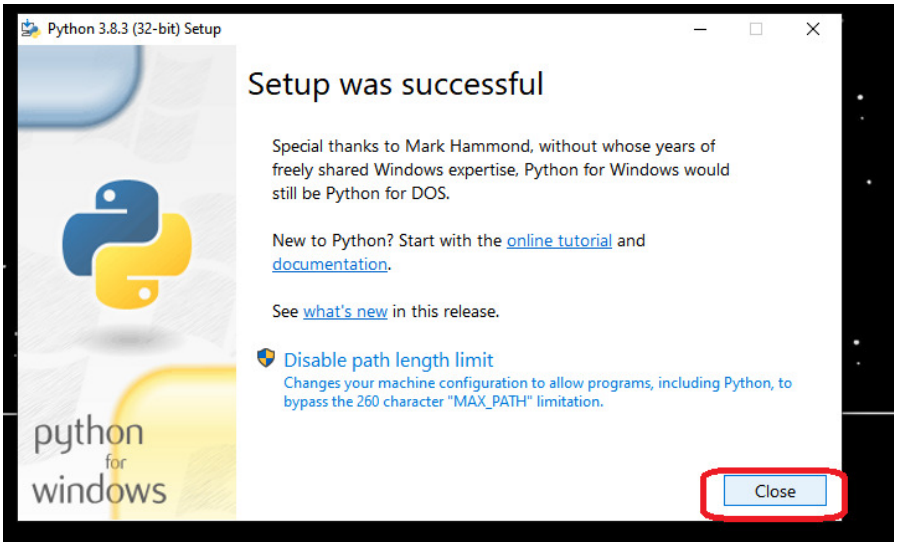

Figura 5: Janela de conclusão da instalação do Python.

2.2 Linux

Se você usa Linux, conforme especificado anteriormente, devemos verificar qual versão do interpretador está instalado no seu computador (no passo a passo a seguir, usamos a distribuição Ubuntu 18.04.02 LTS). Para isso, abra o terminal e digite *python -V* na linha de comando, como pode ser visto na figura abaixo.

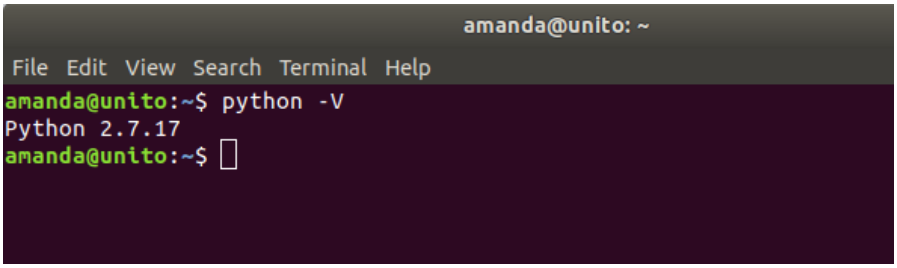

Figura 6: Comando para verificar a versão do interpretador Python.

Caso a versão seja diferente das mais atualizadas (no caso, Python 3.8 ou 3.6), deverá ser instalado a nova versão, como mostra a Figura 7.

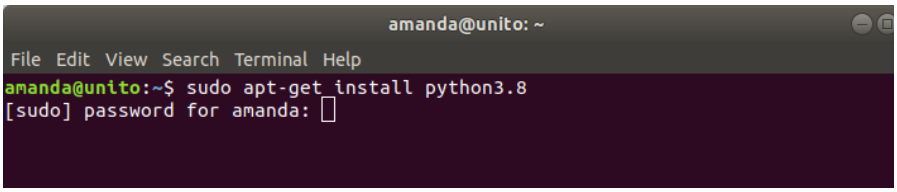

Figura 7: Comando para instalar uma nova versão do interpretador Python.

Ao digitar a senha, surgirá uma mensagem perguntando se você deseja continuar, conforme foi apresentado na Figura 8. Você deverá digitar "y" para continuar.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ sudo apt-get install python3.8
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libpython3.8-minimal libpython3.8-stdlib python3.8-minimal
Suggested packages:
  python3.8-venv python3.8-doc
The following NEW packages will be installed:
  libpython3.8-minimal libpython3.8-stdlib python3.8 python3.8-minimal
0 upgraded, 4 newly installed, 0 to remove and 347 not upgraded.
Need to get 4.551 kB of archives.
After this operation, 18,5 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

Figura 8: Tela de confirmação da instalação.

Quando terminar de carregar, deve-se instalar o IDLE. Então, digite o comando da Figura 9.

```
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ sudo apt-get install idle-python3.8
```

Figura 9: Comando para instalação do IDLE.

Após, aparecerá uma mensagem para continuar, como pode ser visto na figura a seguir. Aperte “y” para terminar a instalação. Caso houver algum erro, você poderá usar outra versão.

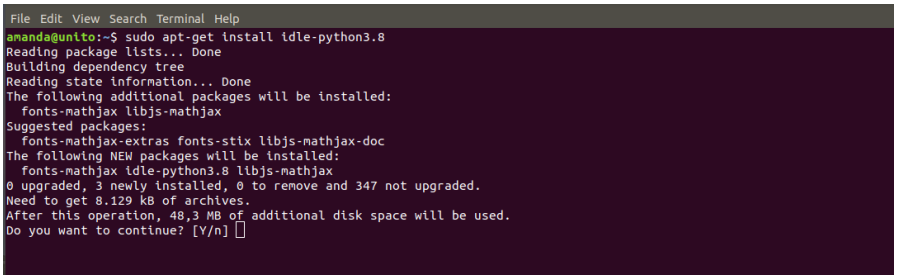

Figura 10: Tela de confirmação da instalação do IDLE.

2.3 Primeiros passos usando o interpretador Python

A IDLE é uma interface gráfica para o interpretador Python, onde nós iremos escrever os nossos códigos. Para abrir o interpretador no sistema operacional Windows deveremos procurar IDLE na lista de aplicativos, como é mostrado na figura abaixo.

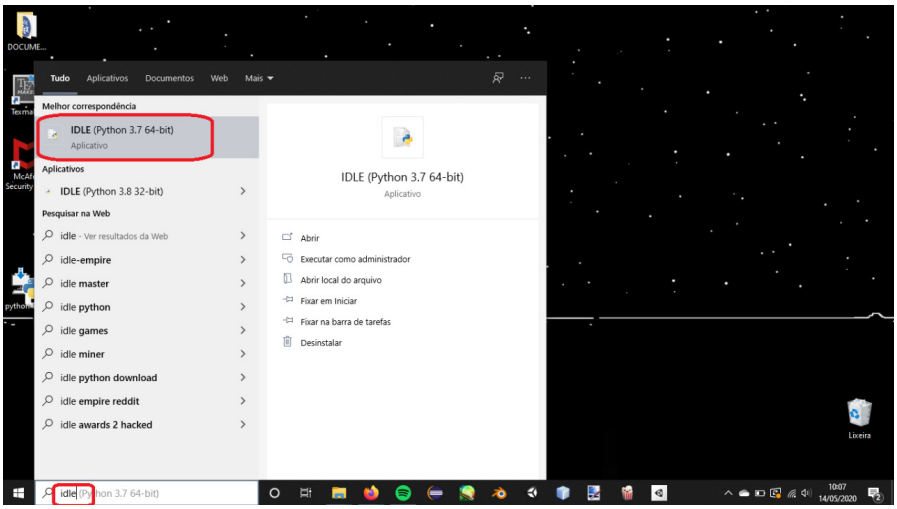

Figura 11: Tela de busca IDLE.

Já para o Linux, devemos digitar o comando apresentado na Figura 12.

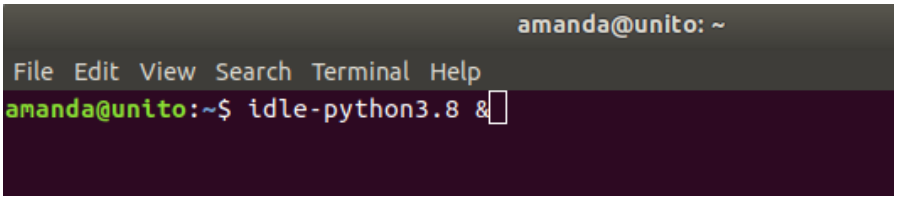

Figura 12: Comando para abrir o IDLE.

Um dos primeiros comandos para se aprender em Python é o de escrever na tela, para isso usaremos o comando "*print*", como pode ser visto na Figura 13.

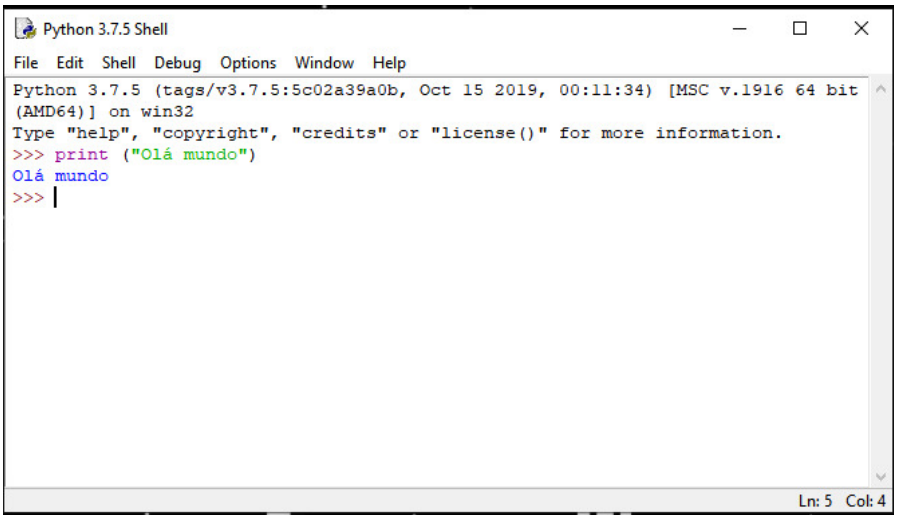

Figura 13: Comando print no IDLE.

Outras intruções básicas para iniciarmos em Python são os de operações matemáticas. Comandos como soma, subtração e adição podem ser vistos na figura abaixo.

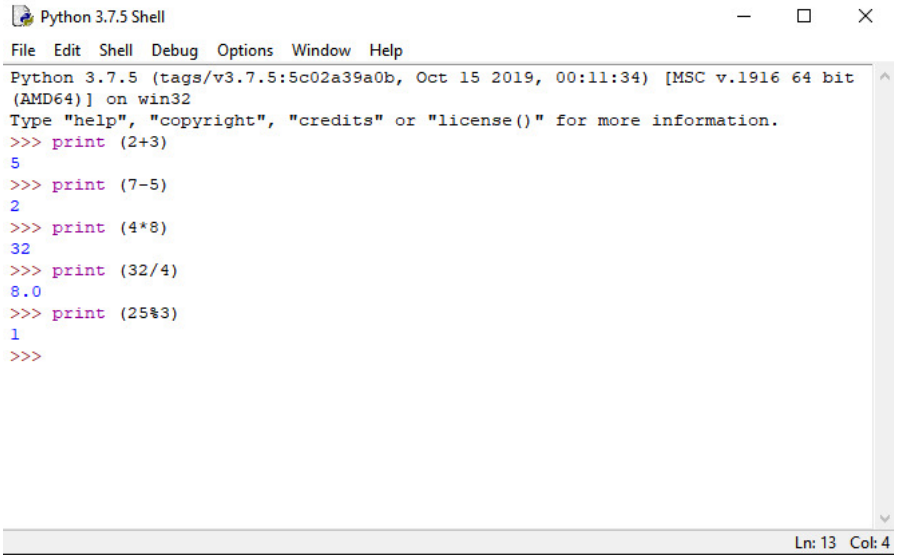

Figura 14: Operações matemáticas no IDLE.

3 Variáveis e Entrada de dados

Neste capítulo vamos definir e exemplificar o que são variáveis e como nomeá-las. Vamos também apresentar o que são operadores relacionais e lógicos e como utilizá-los. Além disso, vamos demonstrar como fazer a leitura de dados do teclado e apresentar as estruturas condicionais da linguagem Python.

3.1 Variáveis

Uma variável pode ser compreendida como qualquer informação que pode assumir diferentes valores. Na programação, resumidamente, podemos definir variáveis como espaços na memória onde são guardadas informações. Essas informações podem ser de diferentes tipos, como valores numéricos (números inteiros e decimais), booleanos (verdadeiro ou falso) e textos. Na tabela abaixo podemos observar os principais tipos de variáveis da linguagem Python.

Tabela 1: Tipos de variáveis em Python.

Tipo	Definição
int	Valores inteiros
float	Valores decimais
double	Valores decimais mais precisos
boolean	Assume os valores true (verdadeiro) ou false (falso)
char	Caractere
String	Caracteres ou Textos

Na Figura 15, são apresentados dois exemplos de variáveis numéricas: a variável "a" e a variável "b". Observe que, ao escrevermos um número decimal, usamos um ponto ao invés da vírgula para separar a parte fracionária. Essa é uma herança da língua inglesa, da qual a maioria das linguagens de programação advém.

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.6.8 (default, Oct  7 2019, 12:59:55)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> a = 3 + 3
>>> print(a)
6
>>> b = 5.34 + 2.77
>>> print(b)
8.11
>>>
```

Figura 15: Exemplos de variáveis numéricas.

Existem também as variáveis do tipo lógico (ou booleano). Esse tipo de variável pode adotar somente dois valores, sendo eles "verdadeiro" e "falso". Podemos observar o exemplo na Figura 16, onde definimos o conteúdo da variável "resultado".

```
>>> resultado = 5 > 4
>>> print(resultado)
True
>>> resultado = 7 < 3
>>> print(resultado)
False
>>>
```

Figura 16: Exemplo de variáveis booleanas.

Ao criarmos uma variável, é preciso ficar atento com o modo como vamos

nomeá-la. Em Python, as variáveis devem começar obrigatoriamente com uma letra e não podem conter espaços em branco, mas podem conter números e sublinha (_) no meio do nome.

3.2 Operadores

Nesta seção vamos definir e apresentar alguns exemplos de operadores relacionais e operadores lógicos.

3.2.1 Operadores Relacionais

Operadores relacionais são utilizados para comparar dois valores do mesmo tipo, onde o resultado da comparação será um valor booleano. Os operadores suportados em Python estão representados na tabela abaixo.

Tabela 2: Operadores Relacionais.

Operador	*Operação*
==	Igualdade
>	Maior que
<	Menor que
>=	Maior ou igual
<=	Menor ou igual
!=	Diferente

A figura abaixo apresenta alguns exemplos de uso dos operadores relacionais.

```
>>> resultado1 = 4 > 7
>>> resultado2 = 2 != 5
>>> print(resultado1)
False
>>> print(resultado2)
True
>>>
```

Figura 17: Exemplos dos operadores relacionais.

3.2.2 Operadores Lógicos

Existem também os operadores lógicos, que possuem o mesmo objetivo dos operadores relacionais, porém comparam valores booleanos. Os principais operadores lógicos são: *not* (não), *and* (e) e *or* (ou).

O operador *not* (não) é um operador unário, ou seja, ele precisa de apenas uma entrada, e sua função é inverter o seu valor. Dessa forma, caso o valor de

entrada for verdadeiro, a saída será falsa, e vice-versa. A Tabela 3 apresenta a tabela verdade desse operador.

Tabela 3: Tabela verdade do operador *not*.

S	*notS*
V	F
F	V

O operador *and* (e) é um operador binário, ou seja, ele precisa de dois valores de entrada. O seu objetivo é comparar os dois valores e, caso ambos forem verdadeiros, a saída será verdadeira. Caso contrário, a saída será falsa. A tabela verdade desse operador é apresentada na Tabela 4.

Tabela 4: Tabela verdade do operador *and*.

S1	S2	S1 and S2
V	V	V
F	V	F
V	F	F
F	F	F

O operador *or* (ou) tem sua tabela verdade representada na Tabela 05. Esse é um operador binário e sua função consiste em comparar dois valores de entrada e, caso algum dos dois valores seja verdadeiro, a saída também será verdadeira.

Tabela 5: Tabela verdade do operador *or*.

S1	S2	S1 or S2
V	V	V
F	V	V
V	F	V
F	F	F

Os operadores lógicos são muito utilizados em expressões lógicas mais complexas, juntamente com os operadores relacionais. A Figura 18 apresenta algumas expressões. Repare que os operadores relacionais são avaliados primeiro, onde retornam respostas booleanas (verdadeiro ou falso) e, só depois, os operadores lógicos são avaliados.

```
>>> 3 > 2 and 4 < 7
True
>>> 8 > 7.5 and 8 < 5
False
>>> 8 > 7.5 or 8 < 5
True
>>>
```

Figura 18: Exemplos de expressões lógicas.

3.3 Leitura de Dados do Teclado

Em Python, para ler dados digitados no teclado pelo usuário, utilizamos a função *input()*. Dessa forma, devemos atribuir o texto digitado em uma variável. Observe o exemplo apresentado na Figura 19, onde o texto digitado pelo usuário foi armazenado na variável "nome".

```
Python 3.7.5 Shell
File  Edit  Shell  Debug  Options  Window  Help
Python 3.7.5 (tags/v3.7.5:5c02a39a0b, Oct 15 2019, 00:11:34) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> nome = input("Digite seu nome: ")
Digite seu nome: Ana
>>> print("Olá, ", nome)
Olá,  Ana
>>>
Ln: 7  Col: 4
```

Figura 19: Exemplo de utilização da função *input()*.

O dado que o usuário fornece e é retornado pelo *input()* é sempre do tipo texto. Para que os valores sejam lidos em formato de números, sejam inteiros ou decimais, é preciso converter o dado recebido. O exemplo de conversão de um texto para um número inteiro pode ser visto na figura abaixo, através da utilização da função *int()*.

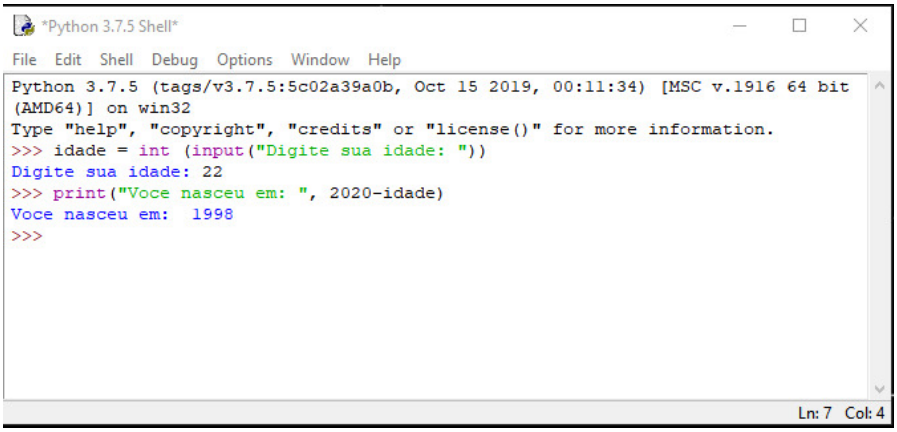

Figura 20: Exemplo de conversão de texto para número inteiro.

3.4 Exercícios

1. Sendo a = False, b = True e c = False, responda os exercícios a seguir apresentando o resultado das suas execuções em Python.

   (a) a and b

   (b) a or b

   (c) not a

   (d) a and c

   (e) b or c

2. Sendo a = 5, b = 20 e c = 12, responda as seguintes questões apresentando o resultado das suas execuções em Python.

   (a) a > b and a < c

   (b) a != c and b > c

   (c) a == b or a < c and b != 20.5

3. Faça um programa que peça dois números inteiros e imprima a soma dos dois na tela.

4. Faça um programa que leia a quantidade de horas e imprima o valor em segundos.

5. Faça um programa que leia o raio de um círculo e imprima sua área e seu perímetro.

4 Condições e Repetições

Neste capítulo vamos definir e exemplificar o conceito de estruturas de condição e repetição.

4.1 Estruturas de Condição

Na maioria das linguagens de programação, assim como em Python, o programa deve ser capaz de decidir se determinada instrução deve ou não ser executada com base em uma condição imposta, ou em valores gerados durante sua execução. Para atender a esse tipo de situação, utilizamos instruções denominadas estruturas condicionais, sendo elas: *if* , *else* e *elif*.

If, traduzindo para o português, significa "se". Este comando é uma estrutura condicional que avalia se tal condição é verdadeira. Se sim, ele executa uma determinada ação. O *else* significa "senão". Se a condição verificada pelo *if* não for verdadeira, a partir do *else* outra determinada ação será executada. Os exemplos de utilização das estruturas condicionais citadas acima podem ser vistos nas figuras a seguir.

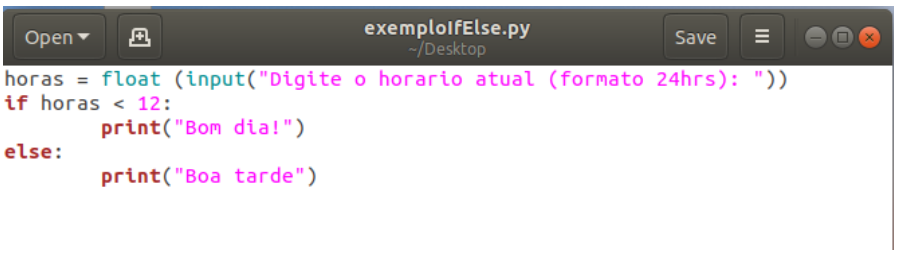

Figura 21: Exemplo da estrutura condicional *if else*.

Observe que a condição imposta está a frente do *if*. Se a entrada do dado, a variável "hora", for um número menor que 12, o programa irá imprimir a mensagem "Bom dia!". Caso contrário, a mensagem imprimida será "Boa tarde.". A Figura 22 mostra a execução do código.

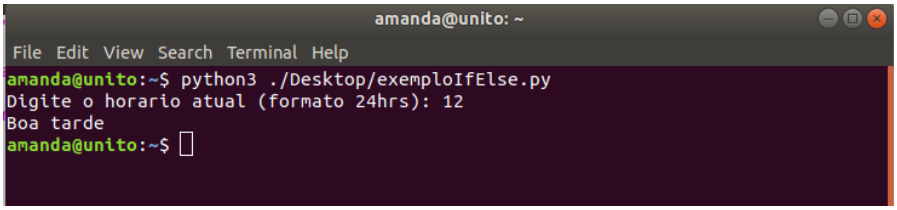

Figura 22: Execução do exemplo da estrutura condicional *if else*.

Já o *elif* significa "senão e se". Se a condição verificada pelo *if* for falsa e se a outra condição imposta for verdadeira, o *elif* executará um comando. Um exemplo da utilização desse comando pode ser visto na figura abaixo.

```
horas = float (input("Digite o horario atual (formato 24hrs): "))
if horas < 12:
        print("Bom dia!")
elif horas>12 and horas<18:
        print("Boa tarde!")
else:
        print("Boa noite")
```

Figura 23: Exemplo da estrutura condicional *if elif else*.

No programa, acima podemos observar que existem duas condições a serem verificadas. Caso a condição imposta no *if* não seja verdadeira, o computador verificará a segunda condição, que é imposta pelo comando *elif*. Caso as condições não forem satisfeitas, o computador executará o comando abaixo do *else*, e imprimirá a mensagem de boa noite. A Figura 24 mostra a execução desse código.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemploIfElseElif.py
Digite o horario atual (formato 24hrs): 22
Boa noite
amanda@unito:~$
```

Figura 24: Execução do exemplo da estrutura condicional *if elif else*.

4.2 Estruturas de Repetição

É comum que em um algoritmo, uma mesma instrução precise ser executada várias vezes seguidas. Para facilitar a programação, normalmente utilizamos um *loop* (ou estrutura de repetição) que permite executar o mesmo bloco de instruções repetidas vezes enquanto uma condição for atendida. Em Python, os laços de repetição podem ser codificados em *while* e *for*.

4.2.1 While

A estrutura de repetição *While* tem como objetivo repetir um comando ou um conjunto de comandos enquanto uma condição for verdadeira. A condição imposta sempre é verificada no início da estrutura e, enquanto for satisfeita, o *While* executa todos os comandos que estão dentro do *loop*. Um exemplo dessa função é apresentado na Figura 25.

```
num = int(input("Informe o ultimo numero a ser impresso: "))
cont = 2

while cont <= num:
        print(cont)
        cont = cont +2
```

Figura 25: Exemplo da estrutura de repetição *While*.

No exemplo acima, utilizamos a estrutura de repetição para a implementação de um contador. Ou seja, o comando incrementa o conteúdo da variável "cont" em duas unidades repetidas vezes, até chegar no valor informado pelo usuário. Dessa forma, o programa imprime todos os números pares até o valor fornecido pelo usuário. A Figura 26 mostra a execução desse código.

```
amanda@unito:~$ python3 ./Desktop/ExemploWhile.py
Informe o ultimo numero a ser impresso: 14
2
4
6
8
10
12
14
amanda@unito:~$
```

Figura 26: Execução do exemplo da estrutura de repetição *While*.

Outra forma muito comum de utilizar a estrutura de repetição é adicionando um acumulador. A diferença de um contador para um acumulador é que, no contador, o valor adicionado é constante. Podemos ver no exemplo da Figura 25, o valor adicionado é sempre dois. Já no acumulador, os valores adicionados são variáveis. A Figura 27 apresenta um exemplo de acumulador, juntamente com o contador.

```
soma = 0          #acumulador
cont = 1          #contador

while cont <= 5:
        n = int(input("Informe um numero inteiro: "))
        soma = soma + n
        cont = cont + 1

print("A soma dos numeros digitados é: %d" %soma)
```

Figura 27: Exemplo da estrutura *While* na implementação de um acumulador.

Observe que existe uma variável "cont", que é responsável por contar quantas vezes o *loop* será executado. A variável "soma" é o acumulador. Ela soma todos os números que o usuário informa e, ao final do *loop*, seu resultado é impresso na tela. A Figura 28 mostra sua execução.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemplo2While.py
Informe um numero inteiro: 2
Informe um numero inteiro: 4
Informe um numero inteiro: 6
Informe um numero inteiro: 3
Informe um numero inteiro: 7
A soma dos numeros digitados é: 22
amanda@unito:~$
```

Figura 28: Execução da estrutura *While* na implementação de um acumulador.

4.2.2 For

O laço *for* é usado quando se tem um bloco de código que será repetido um número fixo de vezes. A estrutura permite percorrer os membros de uma coleção e executar o bloco declarado no *loop*.

```
nomes = ["Ada", "Maria","Cris"]
for n in nomes:
        print(n)
```

Figura 29: Primeiro exemplo da estrutura *For*.

Na Figura 29, pode ser visto um código que percorre uma lista de nomes e imprime todos os seus elementos. Para cada vez que o conjunto de instruções for executado, a variável "n" receberá um item da lista e, quando a lista for inteiramente percorrida, o *loop* terminará. A Figura 30 mostra a execução do código.

```
amanda@unito:~$ python3 ./Desktop/exemplo1For.py
Ada
Maria
Cris
amanda@unito:~$
```

Figura 30: Execução do primeiro exemplo da estrutura *For*.

Como foi dito anteriormente, o comando *for* é executado por um número fixo de vezes. Para isso, em algumas situações, precisamos criar uma variável para contar quantas vezes o bloco de instruções foi executado. No *for*, a forma mais simples de gerar o contador é utilizando a função *range()*. Essa função retorna uma sequência numérica a partir de um intervalo enviado como argumento.

A instrução *range()* pode ser usada de várias formas. Na figura abaixo foi declarado no argumento um único valor, que é chamado de *stop*. Nesse caso o valor de início (ou *start*), será assumido como 0.

```
for i in range(10): #i recebe de 0 a 9, incrementando 1 a cada vez
        print(i) #imprime i
```

Figura 31: Segundo exemplo da estrutura *For*.

Observe que o valor inicial de "i" é zero e, em toda repetição do *loop*, é incrementado um a variável "i". Dessa forma, o programa imprimirá do valor

zero até nove, que é o número informado no argumento da função. A execução desse código é apresentada na Figura 32.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemplo2For.py
0
1
2
3
4
5
6
7
8
9
amanda@unito:~$
```

Figura 32: Execução do segundo exemplo da estrutura *While*.

A função *range()* nos permite também definir o intervalo do *loop*, ou seja, o número inicial e final da repetição, além do *step*, que é o intervalo entre cada elemento.

```
Open ▾   exemplo2.py
         ~/Desktop   Save
for i in range(1,10,2):
        print(i)
```

Figura 33: Segundo exemplo de utilização da função *range()*.

Como pode ser visto na figura acima, o *step* definido foi de 2, então a sequência se dará saltando os valores de 2 em 2. Assim, o programa imprimirá todos os valores ímpares de 1 até 10.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemplo2.py
1
3
5
7
9
amanda@unito:~$
```

Figura 34: Execução do segundo de utilização da função *range()*.

Embora muito úteis, as estruturas de repetições só verificam a condição de parada no início da estrutura. E, dependendo do problema proposto, é interessante podermos finalizar o bloco de comandos no meio. Para isso, utilizamos a função *"break"*, que tem como objetivo terminar de forma abrupta uma repetição. Por exemplo, se estivermos dentro de uma repetição e um determinado resultado ocorrer, o programa deverá sair do *loop* e continuar na primeira linha seguinte à estrutura. Na figura abaixo podemos ver um exemplo de utilização desse comando.

```
soma = 0

for i in range(5):
        n = int(input("Informe um numero inteiro para soma ou 0 para
sair: "))
        if n == 0:
                break
        soma = soma + n

print("A soma dos numeros digitados é: %d" %soma)
```

Figura 35: Exemplo de utilização do comando *break*.

Observe que o comando *"break"* é ativado dentro de uma estrutura de condição. Assim, conseguimos interromper o código dentro do *loop* caso a condição não seja satisfeita. A Figura 36 apresenta a execução desse código.

```
amanda@unito:~$ python3 ./Desktop/exemploBreak.py
Informe um numero inteiro para soma ou 0 para sair: 2
Informe um numero inteiro para soma ou 0 para sair: 4
Informe um numero inteiro para soma ou 0 para sair: 6
Informe um numero inteiro para soma ou 0 para sair: 0
A soma dos numeros digitados é: 12
amanda@unito:~$
```

Figura 36: Execução do exemplo do comando *break*.

4.3 Exercícios

1. Faça um programa que receba dois números e imprima qual dos dois é maior.

2. Faça um programa que receba três números e imprima qual dos três é menor.

3. Faça um programa que receba um ano e determine se ele é bissexto ou não.

4. Faça um programa que mostre ao usuário um menu com 4 opções, sendo elas as operações matemáticas básicas (soma, subtração, multiplicação e divisão). O usuário escolhe uma das opções e o seu programa então pede dois valores numéricos e realiza a operação, mostrando o resultado.

5. Escreva um programa que leia uma data e determine se ela é válida. Ou seja, verifique se o mês está entre 1 e 12, e se o dia existe naquele mês. Lembre-se que Fevereiro tem 29 dias em anos bissextos, e 28 dias em anos não bissextos.

6. Escreva um programa que imprima na tela de 1 até 20, de 1 em 1, 2 vezes. A primeira vez deve usar a estrutura de repetição *for* e a segunda *while*.

7. Faça um programa que leia um número inteiro positivo ímpar N e imprima todos os números ímpares de 1 até N em ordem crescente.

8. Faça um programa que leia 10 inteiros e imprima sua média.

9. Faça um programa que calcule e mostre a soma dos 50 primeiros números pares.

10. Faça um algoritmo utilizando o comando while que mostre uma contagem regressiva na tela, iniciando em 10 e terminando em 0. Mostrar uma mensagem "FIM!"após a contagem.

5 Listas

Listas são um tipo de variável que armazena várias informações ao mesmo tempo. Podemos compará-la com um edifício, que começa pelo térreo, que é o andar zero, em seguida o andar 1, 2, e assim por diante. Os números dos andares são comparados com os índices da lista, que sempre começa pelo zero. Eles servem para especificar em qual posição da lista deseja-se acessar a informação. Os apartamentos são comparados com os dados armazenados naquele índice especificado.

As informações armazenadas nas listas podem ser de vários tipos, sendo elas: textos, caracteres, numeros inteiros, decimais, entre outros. Cada informação é separada por uma vírgula, e todas as informações são armazenadas dentro de um colchete. A Figura 37 apresenta alguns exemplos de listas em Python.

```
v1 = []
v2 = [1,2,3]
v3 = ["oi", 2.3, 3]

print(v1)
print(v2)
print(v3)
```

Figura 37: Primeiro exemplo de listas.

Observe que a lista v1 é uma lista vazia. Ou seja, não possui nenhum valor armazenado nela. Já a lista v2 foi criada com 3 elementos, sendo todos eles números inteiros. Em Pyhton, é possível também criarmos uma lista com mais de um tipo de informação. Como podemos observar, há valores inteiros, decimais e textos armazenados na mesma lista v3. Para imprimir a lista, podemos utilizar o comando *print()*. A Figura 38 mostra a execução desse código.

```
amanda@unito:~$ python3 ./Desktop/exemplo1Listas.py
[]
[1, 2, 3]
['oi', 2.3, 3]
amanda@unito:~$
```

Figura 38: Execução do primeiro exemplo de listas.

Conseguimos também acessar ou modificar cada item de uma lista individualmente. Podemos imaginar um programa onde são armazenadas as notas de todos os alunos. Para modificar algum valor dessa lista, basta informar o nome da lista, junto com o índice da nota que deseja acessar, e igualar à nova nota que deseja armazenar. O índice pode ser comparado como o número correspondente ao aluno. A Figura 39 mostra este exemplo.

```
notas = [6, 5, 7.5, 8, 5.5]

print("Antes de modificar: %d" %notas[1])

notas[1] = 9

print("Depois de modificar: %d" %notas[1])
```

Figura 39: Acessando e modificando itens da lista individualmente.

Lembrando que o índice sempre começa do zero. Ou seja, no exemplo da Figura 39, o número que sofrerá alteração é o segundo número da lista. Veja a execução desse código na Figura 40.

```
amanda@unito:~$ python3 ./Desktop/exemplo2Listas.py
Antes de modificar: 5
Depois de modificar: 9
amanda@unito:~$
```

Figura 40: Exeção do código acessando e modificando itens da lista individualmente.

5.1 Cópia e Fatiamento

Podemos copiar todos os dados de uma lista para outra. Porém, nesse processo devemos ter bastante atenção. Isso porque se somente igualarmos duas listas, não estaremos copiando os dados da lista, e sim seu endereço de memória. Dessa forma, teremos uma mesma lista com dois nomes diferentes.

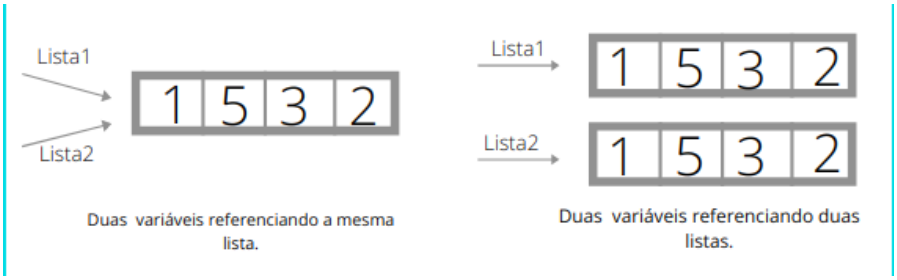

Figura 41: Variáveis referenciando listas.

Felizmente, podemos solucionar este problema de uma forma bem simples. Ao usarmos o comando L[:], é criada uma cópia da lista para um endereço de memória diferente, permitindo que os dados dessas listas sejam modificados de forma independente. A figura abaixo mostra um exemplo de utilização dessa instrução.

```
L = [1, 2, 3, 4, 5]

V = L           # Forma errada
V = L[:]        # Forma certa
```

Figura 42: Criando uma cópia de uma lista.

Também é possível copiar só uma parte de uma lista. Para isso, usamos basicamente o mesmo comando, sendo necessário informar o início e o fim dos índices que se deseja copiar. A Figura 43 mostra o exemplo de utilização desse comando.

```
L = [1, 2, 3, 4, 5]

V = L[:]
V2 = L[2:]      # Criando uma cópia do índice 2 até o final da lista
V3 = L[:3]      # Criando uma cópia até o índice 3

print("Lista V: ")
print(V)

print("\nLista V2:")
print(V2)

print("\nLista V3:")
print(V3)
```

Figura 43: Fatiamento de listas.

Observe que os dados da lista V, a partir do índice 2, são copiados para a lista V2. Já na lista V3, os dados são copiados até o índice 3. Também utilizamos a instrução "" no comando *print()*, que é utilizada para realizar uma quebra de linha no texto. A figura abaixo apresenta a execução desse código.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/ExemploFatiamento.py
Lista V:
[1, 2, 3, 4, 5]

Lista V2:
[3, 4, 5]

Lista V3:
[1, 2, 3]
amanda@unito:~$
```

Figura 44: Execução do código de fatiamento de listas.

5.2 Percorrendo Listas

É possível também calcularmos a nota média da turma usando uma estrutura de repetição. Observe na Figura 45 que usamos a estrutura *for* para percorrer a lista. Porém, também poderíamos calcular usando a estrutura *while*.

```
notas = [6, 5, 7.5, 8, 5.5]
soma = 0

for i in range(5):
        soma = notas[i] + soma

media = soma/5

print("Media das notas: %d" %media)
```

Figura 45: Acessando e modificando itens da lista individualmente.

No código acima usamos duas variáveis. Uma variável "soma", que armazena a soma de todas as notas. E a variável "media", que divide o valor de "soma" por cinco, calculando então a média aritmética das notas. A Figura 46 apresenta a execução desse algoritmo.

```
amanda@unito:~$ python3 ./Desktop/exemplo3Listas.py
Media das notas: 6
amanda@unito:~$
```

Figura 46: Percorrendo uma lista.

Tendo um número fixo de elementos na lista, conseguimos criar uma estrutura que percorra todos os elementos mais facilmente. Como pode ser visto no exemplo acima, só informamos o número de vezes que a estrutura deveria repetir. Porém, em alguns programas não vamos saber essa informação. Para isso, é necessário uma função para descobrir o tamanho da lista. A função que utilizamos em Python é *len()*. Ela retorna o número de elementos de uma lista. A Figura 47 mostra um exemplo de utilização dessa função.

```
L = [1,2,3,4,5]
t = 0

while t < len(L):
        print(L[t])
        t = t+1
```

Figura 47: Percorrendo uma lista.

Nesse caso, a função *len()* foi utilizada para informar quantas repetições seriam necessárias, de acordo com o tamanho da lista. A figura abaixo apresenta a execução do código.

```
amanda@unito:~$ python3 ./Desktop/exemplo2Percorre.py
1
2
3
4
5
amanda@unito:~$
```

Figura 48: Percorrendo uma lista com seu tamanho utilizando a função *len()*.

Podemos também percorrer somente uma parte da lista. Vamos supor que é preciso imprimir os itens de uma lista a partir de um certo índice, ou imprimir só os números com índice par. Para isso, podemos usar a função *range()*. A figura abaixo mostra dois exemplos de utilização dessa função.

```
L = [1,2,3,4,5,6,7,8,9,10]

#primeiro exemplo
print("Imprimindo os números do índice 2 à 9 da lista L:")
for i in range(2,10):
        print(L[i])

#segundo exemplo
print("Imprimindo todos os números pares da lista L:")
for i in range(1,10,2):
        print(L[i])
```

Figura 49: Percorrendo uma lista com a função *range()*.

No primeiro exemplo, utlizamos a função *range()* para imprimir todos os números a partir do índice 2. Já no segundo exemplo, utilizamos para imprimir somente os números pares. O primeiro e segundo número do argumento informam os índices onde a estrutura deve executar. O terceiro argumento, que foi usado somente no segundo exemplo, informa qual o incremento do valor de "i" a cada repetição. A Figura 50 apresenta a execução do código.

```
amanda@unito:~$ python3 ./Desktop/exemploListaRange.py
Imprimindo os números do índice 2 à 9 da lista L:
3
4
5
6
7
8
9
10
Imprimindo todos os números pares da lista L:
2
4
6
8
10
amanda@unito:~$
```

Figura 50: Execução do código que percorre uma lista com a função *range()*.

5.3 Adicionando e Removendo Elementos

Uma das principais vantagens de utilizar-se uma lista é poder adicionar quantos elementos quiser a qualquer hora. Para adicionar um elemento ao final

da lista basta utilizar a função *append()*. Vamos supor que o nome da lista seja L. Assim basta escrever a função dessa forma: *L.append(x)*. O interpretador entenderá que deverá adicionar o valor "x" ao final da lista "L".

A função *append*, apesar de útil para adicionar um elemento ao final da lista, pode causar alguns erros na adição de vários valores ao mesmo tempo. Caso seja necessário a adição de uma lista dentro de outra lista, por exemplo, a função *append()* não será a melhor escolha. Isso porque ao adicionar mais elementos de uma vez, eles ocuparão o mesmo endereço de memória, fazendo com que ocupem o mesmo índice na lista. É possível, porém, adicionar uma lista à outra através da função *extend()*. A figura abaixo mostra exemplos de utilização dessas funções.

```
Open ▾   ExemploAdicaoLista.py   Save
~/Desktop

L = [1,2,3,4,5]
V = [7,8,9,10]

print("Antes de adicionar o número com append():")
print(L)

print("Depois da adição com append():")
L.append(6)
print(L)

print("Antes da instrução extend():")
print(L)

print("Depois da instrução extend():")
L.extend(V)
print(L)
```

Figura 51: Exemplo de adição de elementos em uma lista.

Ao adicionar uma lista com a função *extend()*, cada elemento irá ocupar um espaço de memória diferente, pertencendo a índices também diferentes. A Figura 52 mostra a execução desse código.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/ExemploAdicaoLista.py
Antes de adicionar o número com append():
[1, 2, 3, 4, 5]
Depois da adição com append():
[1, 2, 3, 4, 5, 6]
Antes da instrução extend():
[1, 2, 3, 4, 5, 6]
Depois da instrução extend():
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
amanda@unito:~$
```

Figura 52: Execução do código exemplo de adição de elementos em uma lista.

Podemos também remover dados da lista a qualquer momento. Para isso, podemos utilizar a instrução *del* seguindo do nome da lista e qual o índice do elemento que se deseja remover. Lembrando que, ao remover o elemento da lista, ele não ocupa mais o espaço na memória. Dessa forma, todos os elementos seguintes a ele na lista serão reorganizados com os novos índices. A figura abaixo mostra um exemplo de utilização dessa instrução.

```
Open ▾   exemploDel.py  ~/Desktop   Save
L = [2,4,6,8,10]

print(L)

#Primeiro exemplo
print("Deletando o elemento do índice 2:")
del L[2]
print(L)

#Segundo exemplo
print("\nDeletando os elementos do índice 1 até o índice 3:")
del L[1:3]
print(L)
```

Figura 53: Exemplo de utilização da instrução *del*.

Observe que no exemplo 1 foi removido o elemento do índice 2. Assim, os elementos seguintes a ele, mudaram seus índices. Já no segundo exemplo foi removido todos os elementos entre os índices 1 e 3, restando somente dois elementos na lista. A figura abaixo apresenta a execução desse código.

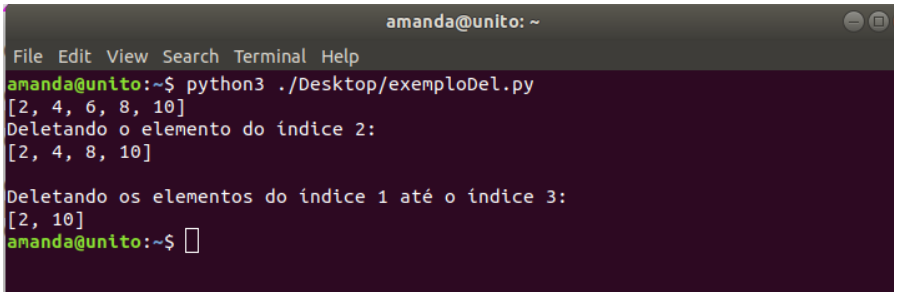

Figura 54: Execução do exemplo de utilização da instrução *del*.

5.4 Exercícios

1. Faça um programa que receba a nota de 7 alunos e calcule a média aritmética da turma.

2. Faça um programa que receba duas listas de 5 elementos cada. Crie uma terceira lista com os elementos das duas primeiras, retirando os valores repetidos.

3. Escreva um programa que leia 10 números inteiros e os armazene em uma lista. Imprima o maior elemento e a posição que ele se encontra.

4. Faça um programa que preencha uma lista com valores informados pelo usuário, excluindo os valores repetidos. O usuário deve escolher quantos números serão armazenados na lista.

5. Faça um programa que receba um número inteiro do usuário. Armazene em uma lista todos os múltiplos desse número.

6 Strings

Uma string é basicamente um grupo de caracteres entre aspas, uma frase ou palavra. Os tipos de dados como *int*, *float* e *bool* são considerados tipos simples ou primitivos, pois seus valores são compostos unicamente por eles, e não por outras partes menores. Já as strings e listas são diferentes, pois elas são construídas de pedaços menores, ou seja, elas podem ser quebradas. No caso de strings, elas são construídas de strings menores, cada uma contendo um caractere.

Os tipos de dados que são compostos por pedaços menores são chamados dados compostos. Dependendo do propósito, podemos tratar esse tipo de dado como composto ou como uma entidade única. Como foi dito anteriormente, podemos definir strings como coleções de sequências de caracteres. Assumimos,

então, que os caracteres que a compõem estão em uma determinada ordem, sendo ela da esquerda para direita.

6.1 Operações com Strings

Em geral, você não pode executar operações matemáticas com strings, mas é possível realizar alguns comandos com os operadores básicos (+ e *). Na figura abaixo, podemos observar como utilizamos o sinal de mais (+) em operações com strings.

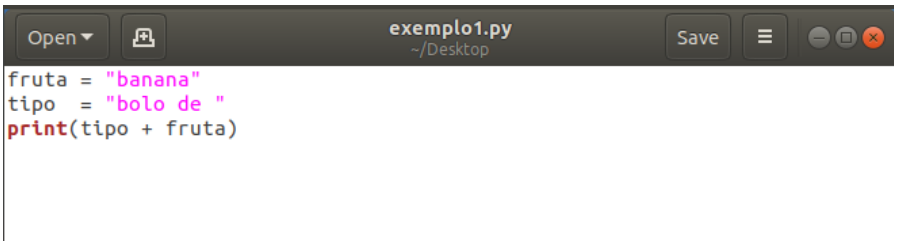

Figura 55: Exemplo de soma de duas strings.

O operador "+" funciona como concatenação de strings. Concatenação significa juntar os dois operandos unindo o fim de um com o começo do outro, como pode ser visto no exemplo da Figura 56.

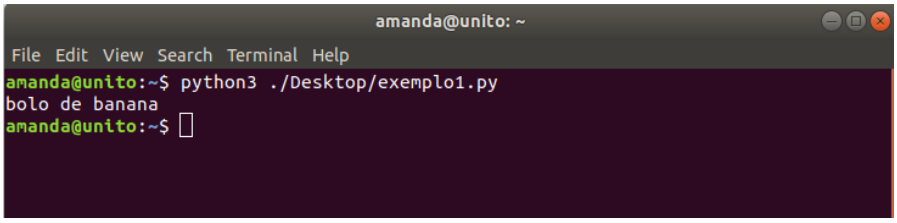

Figura 56: Execução do exemplo de soma de duas strings.

O operador "*" também é usado em operações com strings. Como pode ser visto na Figura 57, ele realiza uma repetição.

```
saudacao = "Bom dia!"
print(saudacao*3)
```

Figura 57: Exemplo de multiplicação de duas strings.

O operador "*" funciona da mesma forma que um multiplicador. Ele multiplica a mensagem o número de vezes que foi especificado na expressão. Na figura abaixo podemos ver melhor como este operador funciona.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemplo2.py
Bom dia!Bom dia!Bom dia!
amanda@unito:~$
```

Figura 58: Execução do exemplo de multiplicação de duas strings.

6.2 Operador de Indexação

Para acessar um caracter dentro da string, utilizamos o operador de indexação. Ele nos permite acessar o caractere por sua posição (valor do índice). Na figura abaixo, podemos entender como funciona a posição de cada caracter pelo valor de cada índice.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P	R	O	J	E	T	O		P	R	O	G	R	A	M	E	N	I	N	A	S
-21	-20	-19	-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Figura 59: Exemplo de operadores de indexação.

Como foi visto na figura acima, as posições também podem ser acessadas da direita para a esquerda, usando números negativos, onde -1 é o índice mais à direita.

Para acessarmos um determinado caractere na string, utilizamos a mesma instrução de acessar um dado na lista. Ou seja, informamos o nome da string, seguido do índice do caractere. A figura abaixo apresenta um exemplo de utilização desse comando.

```
frase = "Projeto PrograMeninas"
primeiro = frase[0]
meio =  frase[10]
ultimo = frase[-1]
nome = frase[8:20]

print (primeiro)
print(meio)
print(ultimo)
print(nome)
```

Figura 60: Exemplo de utilização de operadores de indexação.

Para gerarmos uma string com uma parte da string utilizamos o operador de fatia [n:m]. Ele retorna um pedaço a partir do n-ésimo caractere até o m-ésimo caractere, incluindo o primeiro índice mas excluindo o último. A execução do exemplo do operador de fatia, escrito no código da Figura 60, assim como o acesso a um caractere da string, podem ser visto na figura abaixo.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemplo3.py
P
o
s
PrograMenina
amanda@unito:~$
```

Figura 61: Execução do exemplo de utilização de operadores de indexação.

6.3 Comprimento

Para sabermos o comprimento de um string, utilizamos a função *len()* da mesma forma das listas. Um exemplo de utilização dessa função pode ser visto no exemplo da Figura 62.

```
fruta = "Banana"
print(len(fruta))
```

Figura 62: Exemplo de utilização da função *len* em strings.

Quando falamos de índice começamos a contar do zero. No exemplo da string "Banana", os seis índices são numerados de 0 a 5. Mas quando utilizamos a função *len()*, ela não conta começando do 0, e sim a quantidade de índices da string, como pode ser visto na figura abaixo.

```
amanda@unito:~$ python3 ./Desktop/exemplo4.py
6
amanda@unito:~$
```

Figura 63: Execução do exemplo de utilização da função *len* em strings.

6.4 Comparação de Strings

Para compararmos se duas strings são iguais, utilizamos o operador de igualdade, assim como quando comparamos variáveis numéricas. O exemplo de como comparar duas strings pode ser visto na figura abaixo.

```
queremos = "banana"
fruta = "banana"
if queremos == fruta:
    print("Sim, nos temos bananas!")
else:
    print("Nos não temos bananas.")
```

Figura 64: Exemplo de comparação de duas strings.

A Figura 65 mostra a execução do código acima.

```
amanda@unito: ~
File Edit View Search Terminal Help
amanda@unito:~$ python3 ./Desktop/exemplo5.py
Sim, nos temos bananas!
amanda@unito:~$
```

Figura 65: Execução do exemplo de comparação de duas strings.

6.4.1 Comparadores *in* e *not in*

É possível também verificar se existe um substring dentro de uma string. Para isso, utilizamos o comando *in*. Na Figura 66 podemos observar este comando sendo utilizado.

```
Open ▾   exemplo6.py ~/Desktop   Save
nome = "Marcos"
print('a' in nome)
print('b' in nome)
```

Figura 66: Exemplo de utilização da instrução *in*.

Note que os resultados da função *in* serão dados booleanos (verdadeiro ou falso), como pode ser visto na figura abaixo.

```
amanda@unito: ~
File Edit View Search Terminal Help
amanda@unito:~$ python3 ./Desktop/exemplo6.py
True
False
amanda@unito:~$
```

Figura 67: Execução do exemplo de utilização da instrução *in*.

6.5 Métodos de String

Strings são objetos, e para ligá-las com um método a ser executado, utilizamos a notação de um ponto (“.”). Há uma grande variedade de métodos para serem executados em uma string. Dois deles podem ser vistos na figura abaixo.

```
saudacao = "Ola"
print(saudacao.upper())
cxBaixa = saudacao.lower()
print(cxBaixa)
```

Figura 68: Exemplos de utilização das funções *upper* e *lower*.

Como pode ser visto na Figura 69, *upper* é um método para criar uma nova string no qual todos os caracteres estão em maiúsculo. E o método *lower* funciona de forma semelhante, mas ao invés de tornar os caracteres em maiúsculo, ele torna em minúsculo.

```
amanda@unito:~$ python3 ./Desktop/exemplo7.py
OLA
ola
amanda@unito:~$
```

Figura 69: Execução dos exemplos de utilização das funções *upper* e *lower*.

Além dos métodos citados acima, a tabela a seguir apresenta um resumo de alguns outros métodos que podem ser utilizados em uma string.

Tabela 6: Métodos utilizados em Strings.

Método	**Parâmetros**	**Descrição**
upper	nenhum	Retorna um string todo em maiúsculas
lower	nenhum	Retorna um string todo em minúsculas
capitalize	nenhum	Retorna um string com o primeiro caractere em maiúscula, e o resto em minúsculas
strip	nenhum	Retorna um string removendo caracteres em branco do início e do fim
lstrip	nenhum	Retorna um string removendo caracteres em branco do início
rstrip	nenhum	Retorna um string removendo caracteres em branco do fim
count	item	Retorna o número de ocorrências de item
replace	old, new	Substitui todas as ocorrências do substring old por new
center	largura	Retorna um string centrado em um campo de tamanho largura
ljust	largura	Retorna um string justificado à esquerda em um campo de tamanho largura
rjust	largura	Retorna um string justificado à direita em um campo de tamanho largura
find	item	Retorna o índice mais à esquerda onde o substring item é encontrado
rfind	item	Retorna o índice mais à direita onde o substring item é encontrado
index	item	Como find, mas causa um erro em tempo de execução caso item não seja encontrado
rindex	item	Como rfind, mas causa um erro em tempo de execução caso item não seja encontrado

6.6 Exercícios

1. Escreva um programa que concatene duas strings.

2. Escreva um programa que retorne a string "bOm DiA!" em caixa alta e em caixa baixa.

3. Escreva um programa que receba uma string e após verifique se contém o caractere "a" nela.

4. Escreva um programa que imprima a substring "mundo" da string "Olá mundo!".

5. Escreva um programa que retorne o tamanho da string "projeto Progra-Meninas".

7 Função

Uma função é uma sequência de instruções que computa um ou mais resultados que chamamos de parâmetros. Quando determinadas tarefas precisam ser executadas várias vezes, para que ela não seja escrita diversas vezes durante o codigo, nós agrupamos em um bloco de comandos, que chamamos de função. As funções recebem um nome específico e podem ser chamadas/executadas em diferentes partes do programa.

Ao desenvolver um aplicação, utilizamos funções a todo o momento. Sejam elas as que nós mesmos escrevemos para as rotinas específicas, ou até as disponibilizadas pela própria linguagem de programação, que são disponibilizadas por meio de suas bibliotecas e frameworks. Podemos observar algumas funções disponibilizadas por meios de bibliotecas nos capítulos anteriores, como por exemplo: *print()* e *input()*.

7.1 Sintaxe

Um função é composta por três partes: nome, parâmetro e corpo. O nome é simplesmente como chamamos a função criada. Já o parâmetro é algum dado que passamos da funçao principal para trabalhar na função criada. Por fim, podemos definir o corpo como uma sequência de comandos que representa o comportamento da função. A figura abaixo apresenta um exemplo de função em Python.

Figura 70: Exemplo de utilização de função.

Como podemos observar no exemplo acima, a palavra "*def*" é reservada para definir uma função. Em seguida, informamos o nome escolhido para ela. Ainda na mesma linha, os parênteses servem para indicar os parâmetros. No caso do exemplo acima, a variável "nome". Os dois pontos (:) indicam que o código abaixo faz parte da função. É importante ressaltar que para respeitar a sintaxe da linguagem, os comandos que serão executados precisam estar indentados. Ou seja, as linhas abaixo do cabeçalho da função precisam estar avançados em relação a ele. A figura abaixo apresenta a execução do exemplo da Figura 70.

```
amanda@unito: ~
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemplo1.py
Informe seu nome: Amanda
Bom dia, Amanda !
amanda@unito:~$
```

Figura 71: Execução do exemplo de utilização de função.

Na Figura 71, podemos observar a execução da função "saudacao". No início do programa, foi solicitado o valor da variável "nome" e, em seguida, é apresentado uma mensagem de bom dia com o nome informado pelo usuário. Nas funções, também é possível definir funções com vários ou nenhum argumento, como pode ser visto na figura abaixo.

```
Open ▾   exemplo2.py  ~/Desktop   Save
# Primeira função
def ola():
    print('Olá mundo!')

# Segunda função
def saudacao(nome, horario):
    if(horario<12):
        return ("Olá "+nome+". Bom dia!")
    elif(horario>12 and horario<19):
        return ("Olá "+nome+". Bom tarde!")
    else:
        return ("Olá "+nome+". Bom noite!")

# Chamando as funções
ola()
x = saudacao("Maria", 8)
print(x)
```

Figura 72: Exemplos de utilização de funções com e sem parâmetros.

Na figura acima podemos observar duas funções. A primeira função é denominada "ola" e não possui nenhum parâmetro. Para escrevermos uma função sem parâmetros basta deixar o conteúdo entre os sinais de parênteses em branco. Já na segunda função, "saudacao", podemos notar que há dois parâmetros e que nela possui o comando *return*. A instrução *"return"* tem como objetivo interromper a execução da função e retornar algum valor.

Ao aprendermos sobre funções, é importante também destacar que, se há dois ou mais parâmetros dentro de uma função, é importante ficar atento na ordem em que foram declarados. Isso porque, se temos dois parâmetros, nome

e horário, por exemplo, é preciso passar o conteúdo do nome e depois o horário, respectivamente, caso contrário os valores ficarão invertidos. Na figura abaixo podemos ver a execução das funções citadas na Figura 72.

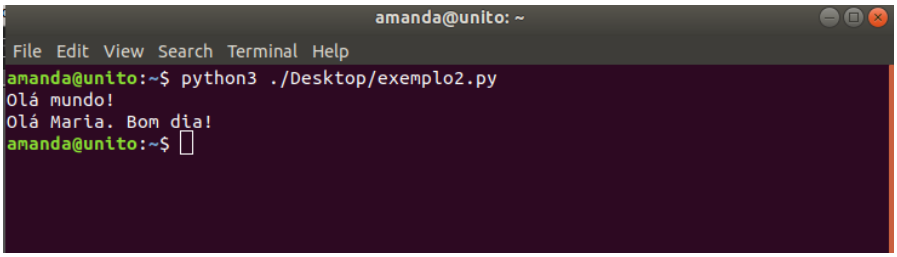

Figura 73: Execução dos exemplos de utilização de funções com e sem parâmetros.

7.2 Exercícios

1. Escreva um programa com uma função soma() que recebe dois números como parâmetros e calcula a soma entre eles.

2. Escreva um programa com uma função subtracao() que recebe dois números como parâmetros e calcula a diferença entre eles.

3. Faça uma função calculadora() que receba dois números como parâmetros e retorne o resultado da soma, subtração, multiplicação e divisão entre eles.

4. Escreva um programa com uma função que receba dois parâmetros, nome e idade, e escreva na tela o nome da pessoa e sua classificação na faixa etária (bebê, jovem, adulto, idoso).

8 Referências

MENESES, N. N. C; **Introdução a Programação com Python:** Algoritmos e lógica de programação para iniciantes. 2. ed. São Paulo: Novatec, 2014.

MANZANO, J. A. N. G; **Introdução a Linguagem Python.** 1. ed. São Paulo: Novatec, 2018.